

ЛАБОРАТОРНАЯ РАБОТА №5

Переопределение методов класса Object в классах табулированных функций

**по курсу
Объектно-ориентированное программирование**

Группа 6204-010302D

Студент: С.О.Куропаткин.

**Преподаватель: Борисов Дмитрий
Сергеевич.**

Задание 1

Переопределение методов в классе FunctionPoint

В классе FunctionPoint были переопределены следующие методы:

```
package functions;

import java.io.*;

public class FunctionPoint implements Serializable {

    private double x;
    private double y;

    // Конструкторы
    public FunctionPoint() { x = 0; y = 0; }

    public FunctionPoint(double x, double y) { this.x = x; this.y = y; }

    public FunctionPoint(FunctionPoint point) { x = point.x; y = point.y; }

    // Геттеры и сеттеры
    public double getX() { return x; }

    public void setX(double x) { this.x = x; }

    public double getY() { return y; }

    public void setY(double y) { this.y = y; }

    public String toString() {
        return "(" + x + "; " + y + ")";
    }

    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        FunctionPoint that = (FunctionPoint) o;
        return (Double.compare(that.x, x)==0 && Double.compare(that.y, y)==0);
    }

    public int hashCode() {
        long combined = Double.doubleToLongBits(x) ^
        Double.doubleToLongBits(y);
        return ((int) (combined ^ (combined >>> 32)));
    }

    public Object clone() {
        return (new FunctionPoint(this));
    }

}
```

Задание 2

Переопределение методов в классе ArrayTabulatedFunction

```

public String toString(){
    String temp = "{";
    for (int i = 0; i < size - 1; ++i){
        temp += getPoint(i).toString() + ", ";
    }
    return temp + getPoint(size - 1).toString() + " }";
}

public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null) return false;

    if (o.getClass() == ArrayTabulatedFunction.class) {
        // Оптимизация для сравнения массивов напрямую
        ArrayTabulatedFunction other = (ArrayTabulatedFunction) o;
        if (this.size != other.size) return false;
        for (int i = 0; i < size; i++) {
            if (!this.points[i].equals(other.points[i])) return false;
        }
        return true;
    }
    if (o.getClass() == LinkedListTabulatedFunction.class){
        for(int i = 0; i < size; ++i)
            if( !( (TabulatedFunction)o).getPoint(i).equals(getPoint(i)) ) )
                return false;
        return true;
    }
    return false;
}

public int hashCode(){
    int hash = 13 * size;
    for(int i = 0; i < size; ++i){
        hash += hash * 3 + getPoint(i).hashCode();
    }
    return hash;
}

public Object clone(){
    FunctionPoint[] temp = new FunctionPoint[size];

    for(int i = 0; i < size; ++i){
        temp[i] = new FunctionPoint((points[i]));
    }
    return (new ArrayTabulatedFunction(temp));
}

```

Задание 3

Переопределение методов в классе LinkedListTabulatedFunction

```

public String toString (){
    String temp = "{";
    for (int i= 0; i < size - 1; ++i){
        temp += getPoint(i).toString() + ", ";
    }
    return (temp + getPoint(size - 1) + " }");
}

public boolean equals(Object o){
    if (this == o) return true;
    if (o == null) return false;

    if (o.getClass() == LinkedListTabulatedFunction.class) {

```

```

        LinkedListTabulatedFunction other = (LinkedListTabulatedFunction) o;
        if (this.size != other.size) return false;

        FunctionNode currentThis = this.head.next;
        FunctionNode currentOther = other.head.next;

        while (currentThis != this.head) {
            if (!currentThis.point.equals(currentOther.point)) return false;
            currentThis = currentThis.next;
            currentOther = currentOther.next;
        }
        return true;
    }
    if (o.getClass() == ArrayTabulatedFunction.class) {
        for (int i = 0; i < size; ++i)
            if ( ! ( (TabulatedFunction)o).getPoint(i).equals(getPoint(i)) )
                return false;
        return true;
    }
    return false;
}

public int hashCode() {
    int hash = 13 * size;
    for (int i = 0; i < size; ++i) {
        hash += hash * 3 + getPoint(i).hashCode();
    }
    return hash;
}

public Object clone() {
    FunctionPoint[] temp = new FunctionPoint[size];
    for (int i = 0; i < size; ++i) {
        temp[i] = new FunctionPoint(getPointX(i), getPointY(i));
    }
    return (new LinkedListTabulatedFunction(temp));
}
}

```

Задание 4

Модификация интерфейса TabulatedFunction

```

package functions;

import java.io.*;

public interface TabulatedFunction extends Function, Cloneable {

    int getPointsCount();
    // Метод, возвращающий ссылку на объект, описывающий точку, имеющую
    // указанный номер
    FunctionPoint getPoint(int index);
    // Метод, заменяющий точку по индексу на заданную
    public void setPoint(int index, FunctionPoint point) throws
    InappropriateFunctionPointException;
    // Метод, возвращающий абсциссу точки по индексу
    double getPointX(int index);
    // Метод, изменяющий абсциссу точки по индексу
    void setPointX(int index, double x) throws
    InappropriateFunctionPointException;
    // Метод, возвращающий ординату точки по индексу
    double getPointY(int index);
}

```

```

// Метод, изменяющий ординату точки по индексу
void setPointY(int index, double y);
// Метод, удаляющий точку по индексу
void deletePoint(int index);
// Метод, добавляющий новую точку
public void addPoint(FunctionPoint point) throws
InappropriateFunctionPointException;
//Метод вывода
void printTabulatedFunction();
public String toString();
public boolean equals(Object o);
public Object clone();
}

```

Задание 5

Тестирование написанных методов

```

import functions.*;
import functions.basic.*;
import functions.meta.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws
InappropriateFunctionPointException {

        // Создание объектов ArrayTabulatedFunction
        double[] values1 = {1, 2, 3, 4, 5};
        ArrayTabulatedFunction arrayFunction1 = new ArrayTabulatedFunction(0,
4, values1);
        ArrayTabulatedFunction arrayFunction2 = new ArrayTabulatedFunction(0,
4, values1.clone());

        // Проверка toString()
        System.out.println("ArrayTabulatedFunction 1: " +
arrayFunction1.toString());
        System.out.println("ArrayTabulatedFunction 2: " +
arrayFunction2.toString());

        System.out.println("Equals (одинаковые точки): " +
arrayFunction1.equals(arrayFunction2));
        System.out.println("Equals (сам с собой): " +
arrayFunction1.equals(arrayFunction1));

        // Проверка hashCode()
        System.out.println("HashCode 1: " + arrayFunction1.hashCode());
        System.out.println("HashCode 2: " + arrayFunction2.hashCode());

        // Проверка clone()
        ArrayTabulatedFunction arrayFunctionClone = (ArrayTabulatedFunction)
arrayFunction1.clone();
        System.out.println("Clone: " + arrayFunctionClone.toString());

        // Изменение оригинала и проверка, что клон не изменился
        arrayFunction1.setPointY(0, 10);
        System.out.println("После изменения: ");
        System.out.println("Original: " + arrayFunction1.toString());
        System.out.println("Clone: " + arrayFunctionClone.toString());
        System.out.println("HashCode Original после изменения: " +
arrayFunction1.hashCode());
        System.out.println("HashCode Clone: " +

```

```

arrayFunctionClone.hashCode());
    System.out.println("Equals (original и clone после изменения): " +
arrayFunction1.equals(arrayFunctionClone));

    // Создание объектов LinkedListTabulatedFunction
    double[] values2 = {1, 2, 3, 4, 5};
    LinkedListTabulatedFunction linkedListFunction1 = new
LinkedListTabulatedFunction(0, 4, values2);
    LinkedListTabulatedFunction linkedListFunction2 = new
LinkedListTabulatedFunction(0, 4, values2.clone());

    // Проверка toString()
    System.out.println("LinkedListTabulatedFunction 1: " +
linkedListFunction1.toString());
    System.out.println("LinkedListTabulatedFunction 2: " +
linkedListFunction2.toString());

    // Проверка equals()
    System.out.println("Equals (одинаковые точки): " +
linkedListFunction1.equals(linkedListFunction2));
    System.out.println("Equals (с самим собой): " +
linkedListFunction1.equals(linkedListFunction1));

    // Проверка hashCode()
    System.out.println("HashCode 1: " + linkedListFunction1.hashCode());
    System.out.println("HashCode 2: " + linkedListFunction2.hashCode());

    // Проверка clone()
    LinkedListTabulatedFunction linkedListFunctionClone =
(LinkedListTabulatedFunction) linkedListFunction1.clone();
    System.out.println("Clone: " + linkedListFunctionClone.toString());

    linkedListFunction1.setPointY(0, 10);
    System.out.println("После изменения:");
    System.out.println("Original: " + linkedListFunction1);
    System.out.println("Clone: " + linkedListFunctionClone);
    System.out.println("HashCode Original после изменения: " +
linkedListFunction1.hashCode());
    System.out.println("HashCode Clone: " +
linkedListFunctionClone.hashCode());
    System.out.println("Equals original и clone после изменения: " +
linkedListFunction1.equals(linkedListFunctionClone));

    // Изменение оригинала и проверка, что клон не изменился
    ArrayTabulatedFunction arraySameAsList = new ArrayTabulatedFunction(0,
4, values2);
    System.out.println("Array и LinkedList (одинаковые точки): " +
arraySameAsList.equals(linkedListFunction2));

    // --- Different data comparison ---
    double[] values3 = {2, 4, 6, 8, 10};
    ArrayTabulatedFunction arrayDifferent = new ArrayTabulatedFunction(0,
4, values3);
    System.out.println("Array (изначальные точки) vs Array (другие точки):
" + arrayFunction2.equals(arrayDifferent));
    }
}

```

Вывод Main:

ArrayTabulatedFunction 1: {(0.0; 1.0), (1.0; 2.0), (2.0; 3.0), (3.0; 4.0), (4.0; 5.0)}

ArrayTabulatedFunction 2: {(0.0; 1.0), (1.0; 2.0), (2.0; 3.0), (3.0; 4.0), (4.0; 5.0)}
Equals (одинаковые точки): true
Equals (сам с собой): true
HashCode 1: -320535552
HashCode 2: -320535552
Clone: {(0.0; 1.0), (1.0; 2.0), (2.0; 3.0), (3.0; 4.0), (4.0; 5.0) }
После изменения:
Original: {(0.0; 10.0), (1.0; 2.0), (2.0; 3.0), (3.0; 4.0), (4.0; 5.0) }
Clone: {(0.0; 1.0), (1.0; 2.0), (2.0; 3.0), (3.0; 4.0), (4.0; 5.0) }
HashCode Original после изменения: 551879680
HashCode Clone: -320535552
Equals (original и clone после изменения): false
LinkedListTabulatedFunction 1: {(0.0; 1.0), (1.0; 2.0), (2.0; 3.0), (3.0; 4.0), (4.0; 5.0) }
LinkedListTabulatedFunction 2: {(0.0; 1.0), (1.0; 2.0), (2.0; 3.0), (3.0; 4.0), (4.0; 5.0) }
Equals (одинаковые точки): true
Equals (с самим собой): true
HashCode 1: -320535552
HashCode 2: -320535552
Clone: {(0.0; 1.0), (1.0; 2.0), (2.0; 3.0), (3.0; 4.0), (4.0; 5.0) }
После изменения:
Original: {(0.0; 10.0), (1.0; 2.0), (2.0; 3.0), (3.0; 4.0), (4.0; 5.0) }
Clone: {(0.0; 1.0), (1.0; 2.0), (2.0; 3.0), (3.0; 4.0), (4.0; 5.0) }
HashCode Original после изменения: 551879680
HashCode Clone: -320535552
Equals original и clone после изменения: false
Array и LinkedList (одинаковые точки): true
Array (изначальные точки) vs Array (другие точки): false